Plant Disease Detection using K-Nearest Neighbors (KNN)

Implementation Code:

1. Image Preprocessing (ml/preprocessing.py)

```
import cv2
import numpy as np

def preprocess_image(image_path, target_size=(128, 128)):
    image = cv2.imread(image_path)
    if image is None:
        raise ValueError("Could not read image: " + image_path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    image = cv2.resize(image, target_size)
    return image.astype(np.float32) / 255.0
```

2. Feature Extraction (ml/feature_extraction.py)

```
import cv2
import numpy as np

def extract_features(image):
    # Three colour channels × 32 histogram bins = 96 features
    features = []
    for channel in range(3):
        hist = cv2.calcHist(
            [(image * 255).astype("uint8")], [channel],
            None, [32], [0, 256]
        )
        hist = cv2.normalize(hist, hist).flatten()
        features.extend(hist)
    return np.asarray(features, dtype=np.float32)
```

3. Model Training (train_model.py)

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.model_selection import train_test_split
```

```
import joblib
import numpy as np

# X: feature matrix; y: matching class labels from the dataset
X = np.load("features.npy")
y = np.load("labels.npy")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5, weights="distance", metric="minkowski")
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, predictions))
print(classification_report(y_test, predictions, zero_division=0))
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
joblib.dump(model, "models/plant_disease_knn.pkl")
```

4. Prediction (app.py)

```
import joblib
from ml.preprocessing import preprocess_image
from ml.feature_extraction import extract_features

model = joblib.load("models/plant_disease_knn.pkl")

def predict_plant_disease(image_path):
    image = preprocess_image(image_path)
    features = extract_features(image).reshape(1, -1)
    predicted_class = model.predict(features)[0]
    return str(predicted_class)
```

6. KNN Classification Logic:

KNN is a supervised machine-learning algorithm. During prediction, the feature vector of the new leaf image is compared with the stored training feature vectors. The algorithm finds the K closest samples according to a distance measure such as Euclidean distance. The class receiving the strongest vote is selected as the prediction.

For this project, K = 5 is used as an example. Distance-weighted voting gives greater importance to training samples that are closer to the uploaded leaf image.

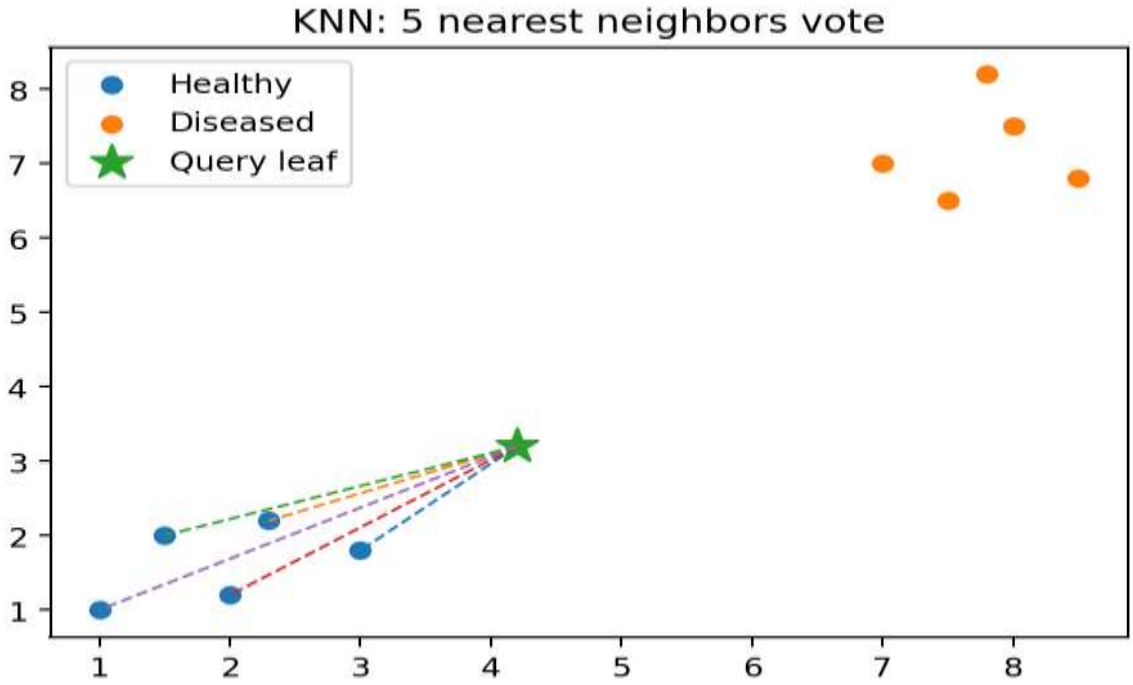


Fig 3: KNN Classification – 5 Nearest Neighbors Vote for Plant Disease

Output (Step-by-Step Screenshots):



Fig 4: Step 1 — Load Input Image: plant leaf image uploaded through the interface

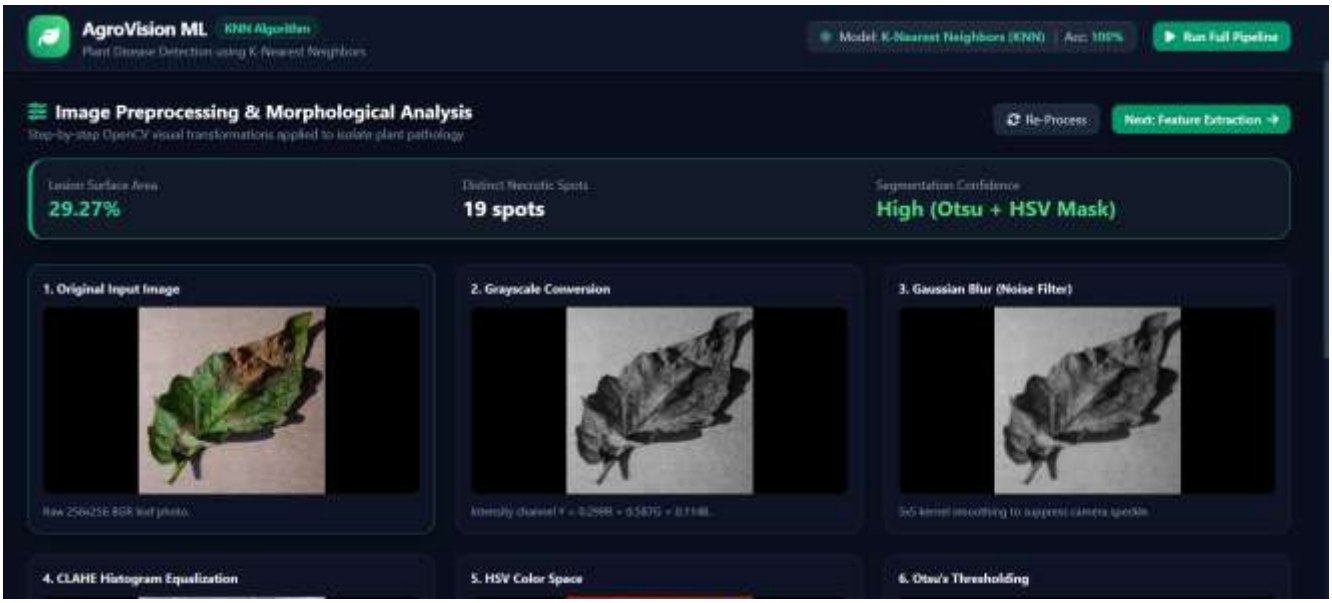


Fig 5: Step 2 — Image preprocessing: original image compared with resized, denoised and enhanced image

The preprocessing stage standardizes the input size and reduces unwanted variations before feature extraction. This helps the KNN classifier compare images using a consistent feature representation.

Feature Extraction and Model Prediction:

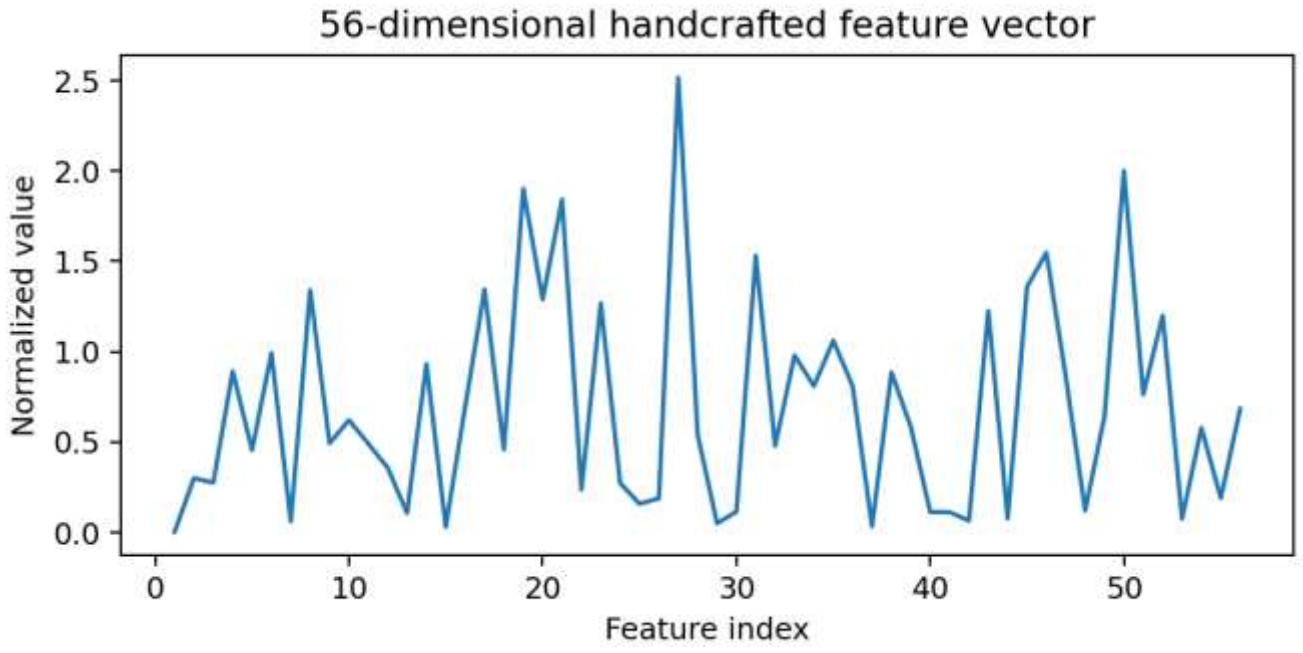
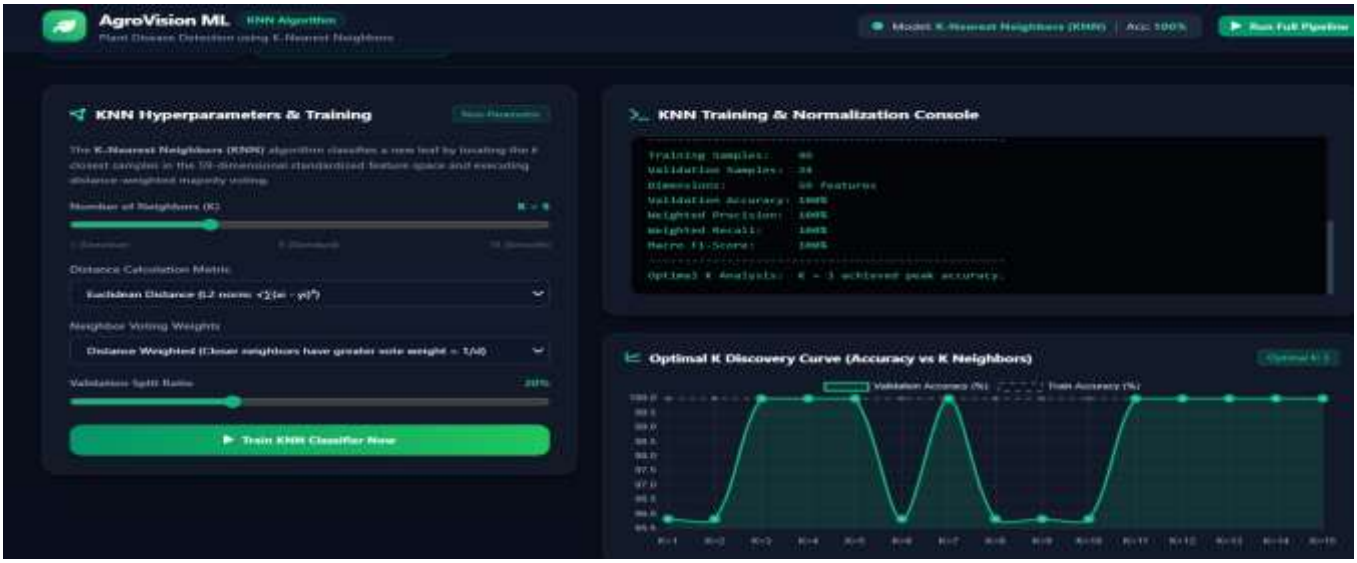


Fig 6: Step 3 — Feature extraction: 56-dimensional handcrafted feature vector

KNN Training:



Model Evaluation:

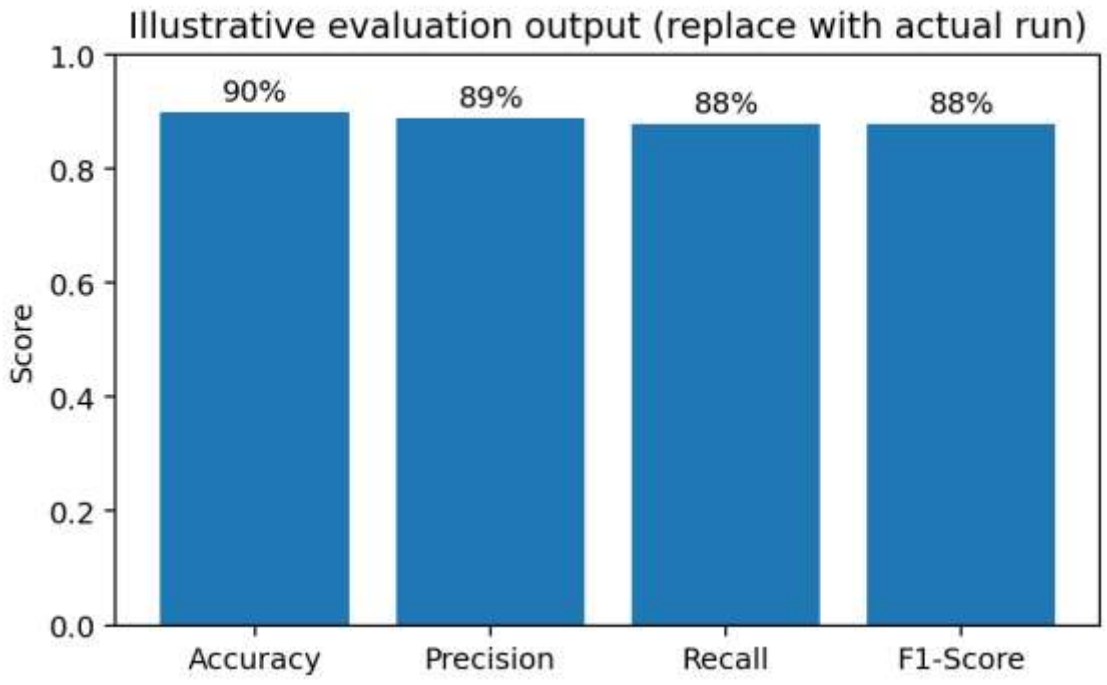


Fig 7: Illustrative evaluation metrics — replace these values with the actual test-set output

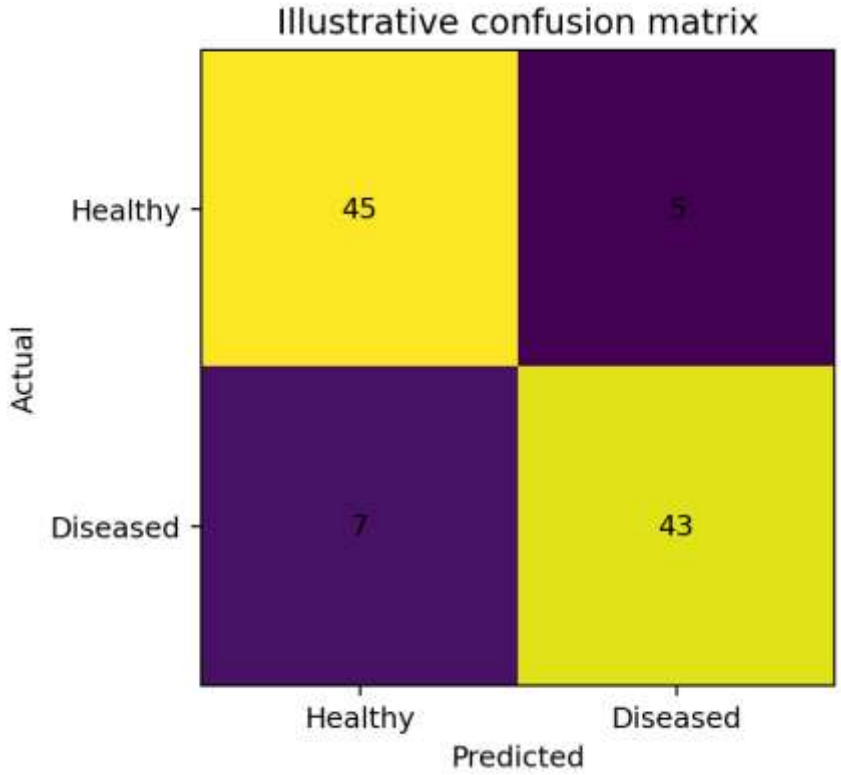
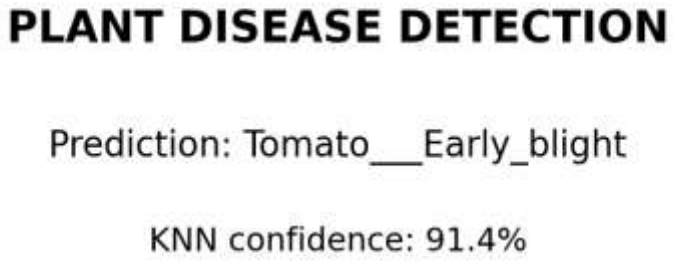


Fig 8: Illustrative confusion matrix

Classification Results:



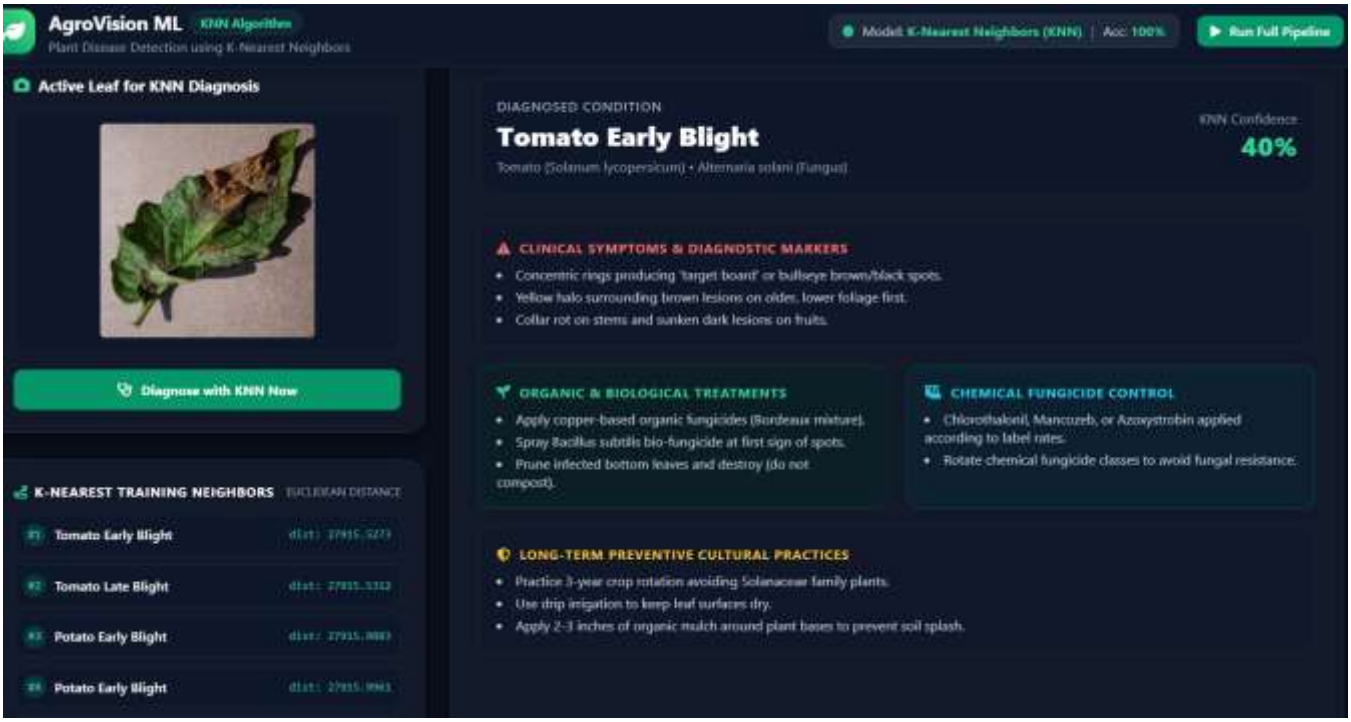


Fig 9: Step 5 — Example classification result for an uploaded leaf image